



Universidad de Oviedo
Grado en Física

Introducción a la Física Computacional

Curso 2023–24

Python 3 como lenguaje algorítmico: Segunda parte

Índice general

5. Python 3 como lenguaje algorítmico: Segunda parte	2
5.1. Objetivo	2
5.2. Entrada y salida estándar de datos	2
5.2.1. Entrada estándar	2
5.2.2. Salida estándar	3
5.2.3. Operador de formato	4
5.3. Estructuras de control de flujo	7
5.3.1. Condicionales	7
5.3.2. Bucles	10
5.3.3. Excepciones	18
5.4. Ficheros	20

Práctica 5

Python 3 como lenguaje algorítmico: Segunda parte

5.1. Objetivo

Familiarizarse con las estructuras de control de flujo y las funciones de entrada y salida.

5.2. Entrada y salida estándar de datos

5.2.1. Entrada estándar

La forma más sencilla de obtener información por parte del usuario es mediante la función `input` (en Python 2 es `raw_input`). Esta función toma como parámetro una cadena a usar como prompt (texto a mostrar al usuario pidiendo la entrada) y devuelve una cadena con los caracteres introducidos por el usuario.

```
nombre = input("¿Como te llamas? ")
print("Encantado, " + nombre)
```

Si necesitamos un valor numérico (incluso una expresión matemática) como entrada en lugar de una cadena utilizaremos la función `eval(cadena)`.

Ejemplo 5.2.1. *Uso de las funciones `input` y `eval`.*

1. *Invoque el modo interactivo de Python y escriba:*

```
>>> a = input('Introduce a: ')
```

al pulsar <Intro> la respuesta será:

Introduce a :

2. Introduzca un número (por ejemplo el 2)

3. Pulse <Intro> y escriba:

```
print(a, type(a))
```

la respuesta será:

```
('2', <class 'str'>)
```

4. Escriba:

```
>>> b = eval(a)
```

```
>>> print(b, type(b))
```

la respuesta es:

```
(2, <class 'int'>)
```

La forma de solicitar una variable numérica es mediante la sentencia:

```
a = eval(input(prompt))
```

5.2.2. Salida estándar

La forma más sencilla de mostrar algo en la salida estándar es mediante el uso de la función `print` (en inglés “imprimir”). En Python 2 `print` es una instrucción y se puede usar también sin paréntesis. Aún así es preferible usarla siempre con paréntesis, ya que así es compatible con Python 3.

En su forma más básica a la función `print` le sigue una cadena de caracteres, que se mostrará en la salida estándar al ejecutarse la sentencia.

```
>>> print('Hola mundo')
```

Hola mundo

La función `str` convierte cualquier objeto en una cadena de caracteres. Podemos producir una salida que combine cadenas de caracteres y variables de distintos tipos usando `str` junto con el operador `+` (que, recordemos, aplicado a 2 cadenas de caracteres, produce una nueva cadena de caracteres formada por las anteriores concatenadas).

La función `print(objeto1, objeto2, ...)` convierte cada objeto en una cadena y los muestra por pantalla en la misma línea separados por espacios. Es posible usar los caracteres especiales para cambiar la forma de mostrar los parámetros por pantalla.

Ejemplo 5.2.2. *Uso de la función `print`.*

1. Invoque el modo interactivo de Python y escriba:

```
>>> a = 1234.56789
```

```
>>> b = [2, 4, 6, 8]
```

```
>>> print(a, b)
```

la respuesta es:

```
(1234.56789 [2, 4, 6, 8])
```

2. Escriba (la función `str` se usa para convertir a cadena):

```
>>> print('a = ' + str(a) + '\nb = ' + str(b))
```

la respuesta es:

```
a = 1234.56789
```

```
b = [2, 4, 6, 8]
```

Nota: se salta línea debido a `\n`.

5.2.3. Operador de formato

En `python` existen diversas maneras de convertir un dato cualquiera en una tira de caracteres que podamos leer los humanos. La manera que aquí veremos es heredada del lenguaje `C` y nos parece suficiente para el uso típico en el campo científico (para números y para tiras de caracteres).

Para especificar el formato de salida de los valores es necesario usar el operador de formato, `%`. Cuando se aplica a enteros, `%` es el operador de módulo, pero cuando el primer operando es una cadena, `%`, es el operador de formato.

El operador `%` puede utilizarse para sustituir los valores de los elementos de una tupla en una cadena de caracteres de acuerdo a un formato (patrón). El resultado es una nueva cadena de caracteres.

La sentencia de conversión es:

```
'%format1 algún texto %format2 ...' % tupla
```

donde la cadena de caracteres a la izquierda del operador contiene un texto salpicado de secuencias como `%format1`, `%format2`, etc. Estas secuencias son las especificaciones de formato de cada objeto dentro de la tupla. El resultado de esta operación es una nueva cadena de caracteres que contiene los valores de las expresiones formateados de acuerdo a los patrones especificados.

Las especificaciones para los formatos más utilizados son:

<code>ws</code>	Cadena
<code>wd</code>	Entero
<code>w.nf</code>	Coma flotante
<code>w.ne</code>	Notación exponencial

donde `w` es la anchura mínima del campo y `n` es el número de dígitos después del punto decimal. Los caracteres `s`, `d`, `f` y `e` se escriben tal cual.

Ejemplo 5.2.3. *Uso del operador de formato.*

1. *Invoque el modo interactivo de Phyton y escriba:*

```
>>> a = 1234.56789
>>> n = 9876
>>> print('%7.2f' % a)
```

la respuesta será:

```
1234.57
```

2. *Escriba:*

```
>>> print('n =%6d' % n)
```

la respuesta será:

```
n = 9876
```

3. *Escriba:*

```
>>> print('n =%06d' % n)
```

la respuesta será:

```
n = 009876
```

4. *Escriba (hacen falta los paréntesis para indicar la tupla):*

```
>>> print('%12.4e%6d' % (a, n))
```

la respuesta será:

```
1.2346e+003
```

```
9876
```

5. *Escriba:*

```
>>> motos = 52
```

```
>>> '%d' % motos
```

la respuesta será:

```
'52'
```

El resultado es la cadena '52', que no debe confundirse con el valor entero 52.

Una secuencia de formato puede aparecer en cualquier lugar de la cadena de formato, de modo que podemos incrustar un valor en una frase.

6. *Escriba:*

```
>>> motos = 52
```

```
>>> 'En julio vendimos%d motos.' % motos
```

La respuesta es:

```
'En julio vendimos 52 motos.'
```

7. En el ejemplo siguiente la secuencia de formato `'%f'` formatea el siguiente elemento de la tupla como un número en coma flotante, y `'%s'` formatea el siguiente elemento como una cadena. Compruébalo escribiendo:

```
>>> 'En%d dias ingresamos%f millones de%s.' % (34,6.1,'euros')
```

La respuesta será:

```
'En 34 dias ingresamose 6.100000 millones de euros.'
```

Por defecto, el formato de coma flotante imprime seis decimales.

El número de expresiones en la tupla tiene que coincidir con el número de secuencias de formato de la cadena. Igualmente, los tipos de las expresiones deben coincidir con las secuencias de formato.

8. Si escribe:

```
>>> '%d%d%d' % (1,2)
```

la respuesta será:

```
TypeError: not enough arguments for format string
```

9. y si escribe:

```
>>> '%d' % 'euros'
```

la respuesta será:

```
TypeError: illegal argument type for built-in operation
```

Para tener más control sobre el formato de los números, podemos detallar el número de dígitos como parte de la secuencia de formato.

10. Escriba:

```
>>> '%6d' % 62
```

```
'62'
```

```
>>> '%12f' % 6.1
```

```
'6.100000'
```

El número tras el signo de porcentaje es el número mínimo de espacios que ocupará el número. Si el valor necesita menos dígitos se añaden espacios en blanco delante del número. Si el número de espacios es negativo, se añaden los espacios tras el número.

11. Escriba:

```
>>> '%-6d' % 62
```

la respuesta (hemos sustituido los espacios por una barra baja para que se vean) será:

```
'62____'
```

También podemos especificar el número de decimales para los números en coma flotante.

12. Escriba:

```
>>> '%12.2f' % 6.1
```

la respuesta será:

'_____6.10'

el resultado ocupa doce espacios e incluye dos dígitos tras la coma. Este formato es útil para imprimir cantidades con las comas alineadas.

Ejercicio 5.2.1. Realice un programa que pida la carga de dos objetos, su distancia e imprima la fuerza entre ambos. El mensaje debe ser claro. La fuerza debe venir expresada en notación científica, con 2 decimales, y se deben especificar todas las unidades (ej. *La fuerza entre los dos objetos de carga QQ.QQ C y QQ.QQ C colocados a una distancia de RR.RR m es de xy.zaE+bc N*)

5.3. Estructuras de control de flujo

Si un programa no fuera más que una lista de órdenes a ejecutar de forma secuencial, una por una, no tendría mucha utilidad. Por un lado, en los programas no existiría la posibilidad de elegir uno de entre varios caminos en función de ciertas condiciones (sentencias alternativas). Y por el otro, no podrían ejecutar algo repetidas veces, sin tener que escribir el código para cada una (sentencias repetitivas).

Para estos dos problemas tenemos dos soluciones: las sentencias de control alternativas y las repetitivas.

Las sentencias alternativas nos permiten seleccionar uno de entre varios caminos por donde seguirá la ejecución del programa. Este tipo de sentencias se denominan **Condicionales**.

Las sentencias repetitivas se denominan también sentencias iterativas—ya que permiten realizar algo varias veces (repetir, iterar)—, y de manera más corriente **Bucles**.

5.3.1. Condicionales

Los condicionales nos permiten comprobar condiciones y hacer que nuestro programa se comporte de una forma u otra, que ejecute un fragmento de código u otro, dependiendo de una condición. Aquí es donde cobran su importancia el tipo booleano y los operadores lógicos y relacionales.

Existen diferentes formas de construir un condicional:

- La forma más simple es un **if** (del inglés "si") seguido de la condición a evaluar, dos puntos (:) y en la(s) siguiente(s) línea(s), e **indentado**, el código a ejecutar en caso de que se cumpla dicha condición, su sintaxis es:

```
if condición:
    código a ejecutar
```

- Una segunda forma de la sentencia **if** es la ejecución alternativa, en la que hay dos posibilidades, y la condición determina cuál de ellas se ejecuta. Su sintaxis es:

```
if condición:
    código a ejecutar
```



```
else:  
    código a ejecutar
```

- La construcción `if ... elif ... elif ... else` se utiliza para condiciones múltiples (`elif` es una contracción de `else if`). Su sintaxis es:

```
if condición:  
    código a ejecutar  
elif condición:  
    código a ejecutar  
else:  
    código a ejecutar
```

Ejemplo 5.3.1. Uso de la construcción `if ... else`.

1. Cree un fichero con el nombre `paridad_if.py` y escriba el siguiente código:

```
'''  
Este programa nos dice si un número es par o impar  
'''  
  
# Pedimos el número cuya paridad queremos conocer  
n = int(input('Dame un número y te diré si es par o impar: '))  
  
# Comprobamos a módulo 2 (el resto de la división de a entre 2)  
# - si es 0 se trata de un número par  
# - en caso contrario se trata de un número impar  
if n%2 == 0:  
    print(str(n) + ' es par')  
else:  
    print('%d es impar'%n)
```

2. Ejecute el programa.

Ejemplo 5.3.2. Uso de la construcción `if ... elif ... elif ... else`.

1. Cree un fichero con el nombre `signo.py` y escriba el siguiente código:

```
'''  
Este programa nos dice si un número es positivo, negativo o nulo  
'''  
  
# Pedimos el número cuya paridad queremos conocer
```

```
x = eval(input('Dame un número y te diré si es par o impar: '))

# Comprobamos el signo. Si no es ni positivo, ni negativo, será nulo
if x < 0.0:
    print('El número %f es negativo'%x)
elif x > 0.0:
    print('El número %f es positivo'%x)
else:
    print('El número es nulo')
```

2. Ejecute el programa.

Asignación condicionada

En ocasiones tenemos que asignar el valor de una variable dependiendo de una condición, como por ejemplo:

```
if x > 3:
    v = 12
else:
    v = 2*x
```

Esto es (aproximadamente) equivalente a la asignación condicionada siguiente:

```
v = 12 if x > 3 else 2*x
```

En general se puede poner:

```
a = b if c else d
```

donde **c** es la condición, **b** es lo que se asigna a la variable **a** si la condición es verdadera, y **d** es lo que se asigna si la condición es falsa. La construcción **b if c else d** puede emplearse en cualquier lugar en el que se espere un valor, por ejemplo:

```
print((x if x>0 else d)*(y if x<0 else -d))
```

Ejercicio 5.3.1. Sea la ecuación de segundo grado $ax^2 + bx + c = 0$. Prepare un código en el que tras pedir los tres coeficientes a , b y c se determine el signo del discriminante $\Delta = b^2 - 4ac$ y según el

mismo indique si la ecuación tiene dos raíces reales, dos raíces complejas conjugadas o una solución real degenerada para a continuación imprimir las soluciones en el formato adecuado.

5.3.2. Bucles

Mientras que los condicionales nos permiten ejecutar distintos fragmentos de código dependiendo de ciertas condiciones, los bucles nos permiten ejecutar un mismo fragmento de código un cierto número de veces, mientras se cumpla una determinada condición. Existen dos maneras de crear bucles: las sentencias `while` y `for`.

Bucles `while`

La sintaxis de la sentencia `while` (en inglés "mientras") es de la forma:

```
while condición:
    código a ejecutar
```

El código se ejecuta mientras la condición sea verdadera. Este proceso continuará hasta que la condición sea falsa.

Ejemplo 5.3.3. *Uso de la construcción `while`.*

1. Cree un archivo con el nombre `crea_lista_w.py` y escriba el siguiente código:

```
'''
Este programa produce una lista con los inversos de los números
naturales usando un bucle while hasta un valor máximo (nMax)
'''

# Número de elementos en la lista
nMax = int(input('¿Cuántos inversos quieres calcular? '))

# Contador
n = 1

# Lista donde almacenaremos los inversos (vacía inicialmente)
inversos = [ ]

# Bucle
while n < nMax:
    inversos.append(1.0/n) # Añade el inverso a la lista
    n += 1 # Incrementamos el contador. Equivalente a n = n + 1

# Imprimimos la lista
```

```
print('La lista con los inversos de los números naturales vale:')  
print(inversos)
```

2. Ejecute el programa escribiendo:

```
python3 crea_lista_w.py
```

3. Observe la respuesta.

Ejemplo 5.3.4. Uso de la construcción `while` junto con `if`.

1. Cree un fichero con el nombre `escribe_pares.py` y escriba el siguiente código:

```
'''  
Este programa produce una lista con los números pares  
menores que un valor dado (nMax)  
'''  
  
# Número máximo de valores a considerar  
nMax = int(input('¿Hasta dónde quieres buscar números pares? '))  
  
# Contador  
i = 1  
  
# Lista donde almacenaremos los números pares  
pares = []  
  
# Bucle  
while i < nMax:  
    if i%2 == 0:          # si el resto de la división por 2 es 0 (es par)  
        pares.append(i) # añade un elemento a la lista  
        i += 1  
  
print('La lista de números pares menores que %d es: '%nMax)  
print(pares)
```

2. Ejecute el programa.

Al igual que con la construcción `if`, con la sentencia `while` también podemos utilizar más de dos ramas usando el condicional encadenado, cuya sintaxis es:

`while condición:`

`código a ejecutar`

else:

código a ejecutar

En este caso, el código en el bloque `else` se ejecuta cuando la *condición* deja de ser válida. Esta estructura puede ser útil en combinación con la sentencia `break` que veremos más adelante.

Ejercicio 5.3.2. Genere la sucesión de números entre 1 y n (que se pedirá) que sean a la vez múltiplos de 11 y 13, imprimiendo término a término.

Ejercicio 5.3.3. Modifique el programa anterior para que guarde los múltiplos de 11 y 13 en una lista, e imprima la lista al final.

Bucles `for`

La construcción `for ... in` (en inglés "para ... en") también nos permite realizar bucles y su sintaxis es:

`for e in S:`

código a ejecutar

Esta sentencia irá asignando en cada iteración el valor de cada uno de los elementos de la secuencia `S` a la variable `e` y ejecutando las líneas de código que hay en el bucle. En Python `for` se utiliza para recorrer secuencias (listas, tuplas o cadenas). En Python 3 la secuencia `S` además puede ser un objeto iterable.

Ejemplo 5.3.5. *Uso de la construcción `for ... in`.*

1. Cree un archivo con el nombre `bluce_for.py` y escriba el siguiente código:

```
'''
Este programa itera sobre los elementos de una lista de números
mediante un bucle for para calcular sus inversos.
'''

# Construimos una lista con unos cuantos elementos
numeros = [1, 7, 33, 56]

# Lista vacía donde guardaremos los inversos
inversos = [ ]

# Iteramos sobre la lista calculando sus inversos y añadiéndoselos
# a la lista de inversos
for elemento in numeros:
    inversos.append(1.0/elemento)
```

```
# Imprimimos la lista
print('Dados los siguientes números:')
print(numeros)
print('Sus inversos valen:')
print(inversos)
```

2. Ejecute el programa y observe la respuesta.

Algunos iterables interesantes: `range` y `enumerate`

En muchas ocasiones la iteración de un bucle `for` se realiza sobre un objeto de tipo *rango* (`range`). Un *rango* es un objeto iterable, de tal manera que en cada paso del bucle va tomando diversos valores enteros: es como si tuviéramos una tupla cuyos valores se van generando sobre la marcha (esto es más eficiente en el manejo de la memoria para el caso de rangos muy grandes). La forma más general de obtener un *rango* es usar la función (constructor) siguiente:

```
range(inicio, final, paso)
```

donde:

- `inicio`: Un `int` que indica el primer valor (incluido) del rango.
- `final`: Un `int` que indica el último valor (no incluido) del rango.
- `paso`: Un `int` que indica el tamaño del paso para generar el próximo número de la secuencia

Para facilitar los casos más comunes, `range` tiene constructores con menos parámetros. Así, `range(f)` iterará desde 0 hasta `f-1` de 1 en 1 ($0, 1, 2, \dots, f-1$). Mientras que `range(i, f)` iterará desde `i` hasta `f-1` de 1 en 1 ($i, i+1, i+2, \dots, f-1$).

Si queremos una lista con los valores devueltos por `range` debemos convertirlo explícitamente a una lista:

```
L = list(range(4))    # -> [0, 1, 2, 3]
```

Otro iterador ya comentado en la práctica anterior es `zip`.

Ejemplo 5.3.6. Uso de la construcción `for ... in`.

1. Cree un archivo con el nombre `crea_lista_f.py` y escriba el siguiente código:

```
'''
Este programa produce una lista con los inversos de los números
naturales usando un bucle for hasta un valor máximo (nMax)
'''

# Número de elementos en la lista
nMax = int(input('¿Cuántos inversos quieres calcular? '))
# Lista donde almacenaremos los inversos (vacía inicialmente)
inversos = [ ]

# Bucle for usando range hasta nMax + 1 para incluir nMax
for n in range(1, nMax + 1):
    print("En esta iteración n vale " + str(n))
    inversos.append(1.0/n)

# Imprimimos la lista
print('La lista con los inversos de los números naturales vale:')
print(inversos)
```

Aquí la secuencia a iterar es 1, 2, ..., nMax, que hemos creado llamando al objeto iterable `range`.

2. Ejecute el programa y observe la respuesta.

Se puede iterar en una lista mediante el *índice* y el *elemento* simultáneamente usando la función de lista `enumerate`.

Ejemplo 5.3.7. Uso de la función `enumerate`.

1. Cree un archivo con el nombre `prueba_enumerate.py` y escriba el siguiente código:

```
'''
Este ejemplo itera simultáneamente sobre los elementos y los índices
de una lista usando enumerate
'''

# Construimos una lista con unos cuantos elementos
mi_lista = ['a', 5, (1.0, 2.0)]

# Iteramos sobre la lista mostrando cada elemento y el índice asociado
for indice, elemento in enumerate(mi_lista):
    print('El elemento ' + str(indice) + ' vale ' + str(elemento))
```

2. Ejecute el programa y compruebe el resultado.

Comprensión de listas

En matemáticas se usa una descripción de conjuntos en la forma:

$$A = \{x \mid \text{cond1}, \text{cond2}, \dots\},$$

donde A es el conjunto formado por los valores x que cumplen las condiciones cond1 , cond2 , etc.

En `python` existe una construcción, denominada «comprensión de listas» (en inglés *list comprehension*) que permite casi este formato, pero el conjunto A debe ser finito y numerable. En las diversas condiciones se admite la inclusión, `in`, y el condicional, `if`.

Ejemplo 5.3.8. *Lista con cuadros de números pares (n es conocido).*

$$A = \{x^2 \mid x \in \{0, \dots, n\}, x \text{ es par}\},$$

que en `python` sería:

```
A = [x**2 for x in range(n+1) if x % 2 == 0]
```

Como vemos:

- el conjunto $\{ \}$ se «convierte» en lista `[ ]`;
- «tal que», $\mid$, se «convierte» en uno o varios espacios (luego va siempre `for`);
- la inclusión, o sea $\in$, se «convierte» en `for ... in` (se pueden repetir);
- la coma «, $\rangle$ se «convierte» en `if` (se pueden repetir).

Por otro lado el conjunto $\{0, \dots, n\}$ se genera en `python` con el iterador `range`, pero podría ser cualquier lista ya conocida en ese punto del código.

Ejemplo 5.3.9. *Lista con inversos de los números naturales.*

1. Cree un archivo con el nombre `crea_lista_fc.py` y escriba el siguiente código:

```
'''
Este programa produce una lista con los inversos de los números
naturales hasta un valor máximo (nMax) usando comprensión de listas
'''

# Número de elementos en la lista
nMax = int(input('¿Cuántos inversos quieres calcular? '))
```



```
# Generamos la lista inversos por comprensión de listas
inversos = [1.0/n for n in range(1, nMax+1)]

# Imprimimos la lista
print('La lista con los inversos de los números naturales vale:')
print(inversos)
```

2. Ejecute el programa y compruebe el resultado.

Ejercicio 5.3.4. Prepare un programa que genere lista con los números menores que n (que se pide por teclado) que sean a la vez múltiplos de 11 y 13, o sea, prepare el conjunto B que verifique:

$$B = \{x \mid x \in \{1, 2, \dots, n\}, x \text{ es divisible por } 11, x \text{ es divisible por } 13\},$$

Ejemplo 5.3.10. Las ternas pitagóricas que son tripletas de números enteros positivos tales que la suma del cuadrado de dos de ellos nos da el cuadrado del tercero, o sea:

$$T = \{(x, y, z) \mid x \in \{1, \dots, n\}, y \in \{1, \dots, n\}, z \in \{1, \dots, n\}, x^2 + y^2 = z^2\},$$

que se puede poner:

```
T = [(x,y,z) for x in range(1,n+1) for y in range(1,n+1)
      for z in range(1,n+1) if x**2+y**2 == z**2]
```

Ejemplo 5.3.11. Si la variable que se asigna como elemento de lista depende de una condición doble debemos usar la asignación condicional, por ejemplo:

```
V = [(x,y) if x % 2 == 0 else (y,x) for x in range(n) for y in range(n)]
```

Ejercicio 5.3.5. Cree una lista con los números enteros de 1 hasta n (que es un dato) y luego modifique los términos múltiplos de 3 cambiándolos de signo. Finalmente imprima la lista final para comprobar que el programa funciona bien.

Cómo alterar los bucles: `break` y `continue`

La sentencia `break` sirve para finalizar un bucle, independientemente de que la condición del `while` aún se cumpla o de que aún queden elementos en la secuencia de un `for`.

Ejemplo 5.3.12. En el ejemplo siguiente, para obtener lo que el usuario escriba en pantalla utilizamos la función `input`.

1. Cree un archivo con el nombre `lista_nombres.py` y escriba el siguiente código:

```
'''
Este programa pide al usuario un nombre y lo compara con una lista de
nombres ya existentes. El programa informa de la posición que ocupa el
nombre introducido en la lista existente si lo encuentra. En caso
contrario muestra un mensaje informando del fracaso de la búsqueda
'''

# Lista de nombres ya existentes
lista_nombres = ['Marta', 'Juan', 'Pedro', 'Carmen']

# Pedimos al usuario un nombre
nombre = input('Escribe un nombre: ')

# Esta variable será False mientras no encontremos el nombre en la
# lista de nombres
encontrado = False

# Iteramos a los índices de la lista de nombres comparando cada uno de
# sus elementos con el nombre que pedimos
for i in range(len(lista_nombres)):
    if lista_nombres[i] == nombre:
        # Si los nombres coinciden, mostramos un mensaje, cambiamos
        # el valor de la variable 'encontrado' a True, y finalizamos
        # el bucle para no perder más tiempo
        print(nombre + ' es el número ' + str(i+1) + ' de la lista')
        encontrado = True
        break

# Si la variable 'encontrado' sigue siendo False significa que no
# encontramos el nombre que introdujo el usuario en la lista de nombres
if not encontrado:
    print(nombre + ' no está en la lista')
```

2. Ejecute el programa y observe la respuesta.

Por otro lado, la sentencia `continue` salta directamente a la siguiente iteración del bucle.

Ejemplo 5.3.13. Uso de la sentencia `continue`.

1. Crea un archivo con el nombre `lista_div7.py` y escriba el siguiente código:

```
'''
Este programa construye una lista con todos los números entre nMin y
nMax que no sean ni múltiplos de 7, ni números pares
'''
```

```
'''  
  
# nMin y nMax indican desde dónde hasta dónde vamos a buscar  
nMin, nMax = 10, 50  
  
# Lista donde almacenaremos los números (vacía inicialmente)  
numeros = [ ]  
  
# Iteramos a los valores que queremos  
for i in range(nMin, nMax + 1):  
    # Si es múltiplo de 2 o de 7, seguimos con la siguiente iteración  
    if i % 2 == 0: continue  
    if i % 7 == 0: continue  
  
    # Si hemos llegado aquí es porque no es ni múltiplo de 2, ni de 7  
    print('Hemos encontrado un número válido: ' + str(i))  
    numeros.append(i)  
  
print('La lista final de número es:')  
print(numeros)
```

2. Ejecute el programa y observe la respuesta.

Ejercicio 5.3.6. Cree un fichero con el nombre `TemperaturasFC.py` que calcule las temperaturas en la escala Fahrenheit equivalentes al rango de temperaturas desde -20°C hasta 40°C con saltos de 5°C . Para ello utilice un bucle `while` para crear dos listas `Fdeg` y `Cdeg` que almacenen respectivamente los valores de las temperaturas en grados Fahrenheit y en grados Celsius. La fórmula para la conversión es $F = \frac{9}{5}C + 32$.

Ejercicio 5.3.7. Repita el ejercicio anterior utilizando un bucle `for` en lugar de `while`. Además solicite que el rango de temperaturas en $^{\circ}\text{C}$ considerado y el tamaño de los saltos sea introducido por teclado. Guarde el programa como `TemperaturasFCv1.py`.

Ejercicio 5.3.8. Modifique el programa `TemperaturasFCv1.py` de manera que los valores de las temperaturas se muestren en una tabla donde la primera columna represente las temperaturas en $^{\circ}\text{C}$ con formato entero y en la segunda columna se muestren los valores correspondientes en $^{\circ}\text{F}$ con una cifra decimal. Guarde el nuevo programa con el nombre `TemperaturasFCv2.py`.

5.3.3. Excepciones

La sentencia `try` (en inglés "intentar") permite controlar errores de ejecución sin que el programa aborte. Su sintaxis es la siguiente:

```
try:  
    sentencias que se prueban
```

```
except:

    sentencias que se ejecutan en caso de error

else: # opcional

    sentencias que se ejecutan si no hay error

finally: # opcional

    sentencias que se ejecutan siempre al final
```

Por otro lado, podemos hacer abortar un programa usando la sentencia `assert` (podríamos traducirlo por "comprobar") que comprueba la veracidad de una condición de tal modo que si es falsa nos muestra un mensaje de error y aborta el programa.

Ejemplo 5.3.14. Uso de excepciones.

1. Cree un archivo con el nombre `excep.py` y escriba el siguiente código:

```
'''
Este programa pide un número e intenta hallar su inverso. Si el número
introducido es 0, el programa en lugar de fallar muestra un mensaje.
Además exigimos que el número sea no negativo.
'''

# Pedimos un número
x = eval(input('Dame un número cualquiera y te diré su inverso: '))

# Si queremos asegurarnos de que el número sea no negativo podemos usar
# una sentencia assert
assert x >= 0, 'El número debe ser no negativo'

# Ahora intentamos calcular el inverso
try:
    inverso = 1/x
except:
    print('Problemas! División por cero')
else:
    print('El inverso de ' + str(x) + ' vale ' + str(inverso))
finally:
    print('esto se ejecuta siempre al final')
```

2. Ejecute el programa y vea la salida usando diferentes entradas enteras: positivas (deberían funcionar sin problemas), 0 (que dará un mensaje de error) y negativas (el programa abortará).

5.4. Ficheros

Mientras un programa se está ejecutando, sus datos están en la memoria. Cuando el programa termina, o se apaga el computador, los datos de la memoria desaparecen. Para almacenar los datos de forma permanente se deben poner en un archivo o fichero. Antes de poder escribir en un fichero o acceder a los datos del mismo, se debe crear un objeto fichero con el comando `open` (en inglés “abrir”):

```
objeto_fichero = open(nombre_fichero, acción)
```

siendo `nombre_fichero` la cadena que especifica el fichero que va a ser abierto, incluido su camino de búsqueda (`path`) si es necesario. La `acción` es una de las siguientes cadenas:

Acción	Definición
'r'	Leer de un fichero existente (se usa por defecto si no se especifica la acción)
'w'	Escribir en un fichero. Si el fichero no existe lo crea, si existe lo sobrescribe.
'a'	Añade al final de un fichero.
'r+'	Lee y escribe en un fichero existente.
'w+'	Lo mismo que 'r+', pero si el fichero no existe, lo crea.
'a+'	Lo mismo que 'w+', pero los datos son añadidos al final del fichero.

Es una buena práctica de programación cerrar el fichero si no se va a acceder a él durante un tiempo largo dentro del programa. Para ello ejecutamos el comando:

```
objeto_fichero.close()
```

Existen varias maneras de leer datos de un fichero:

- `s = objeto_fichero.read(n)`
Lee `n` caracteres y los devuelve como una cadena `s`. Si se omite `n`, se leerán todos los caracteres del fichero. Se puede crear una lista con todas las líneas usando:
`s.splitlines()`
- `objeto_fichero.readline(n)`
Lee `n` caracteres de una línea y los devuelve como una cadena. Los caracteres los devuelve como cadena que termina con `\n`. Si se omite `n` se lee toda la línea.
- `objeto_fichero.readlines()`
Devuelve una lista de las líneas desde la primera línea a la última línea del fichero.

También existen diferentes maneras de escribir en un fichero:

- Para escribir una cadena:
`objeto_fichero.write()`

- Para escribir una lista de cadenas:

```
objeto_fichero.writelines()
```

Ninguno de los métodos añade el carácter `'\n'` al final de la línea.

- Utilizando la función `print` para escribir en un fichero de salida:

```
print(objeto1, objeto2, ..., file=objeto_fichero)
```

La función `print` siempre añade el carácter `'\n'` al final de la línea.

Las funciones `read`, `readline`, `readlines`, `write`, `writelines` y otras que se usan de la manera:

```
objeto.funcion(...)
```

se denominan **métodos** asociados al objeto (es una notación de la programación orientada a objetos).

Ejemplo 5.4.1. Abrir y cerrar ficheros.

1. Invoque el modo interactivo de Python y escriba:

```
>>> f = open('datos.dat', 'w')
```

```
>>> print(f)
```

la respuesta será:

```
<_io.TextIOWrapper name='datos.dat' mode='w' encoding='UTF-8'>  
(o algo similar)
```

La función `open` toma dos argumentos. El primero es el nombre del archivo y el segundo es el modo. El modo `'w'` significa que lo estamos abriendo para escribir. Si no hay un archivo llamado `test.dat` se creará. Si ya hay uno, el archivo que estamos escribiendo lo reemplazará. Al imprimir el objeto archivo, vemos el nombre del archivo, el modo y la localización del objeto.

2. Para cerrar el fichero escriba:

```
>>> f.close()
```

3. Si se intenta abrir un fichero que no existe, se recibe un mensaje de error. Escriba:

```
>>> f = open('medidas.dat', 'r')
```

la respuesta será:

```
Traceback (most recent call last):  
File <stdin>, line 1, in <module>  
FileNotFoundError: [Errno 2] No such file or directory: 'medidas.dat'
```

Ejemplo 5.4.2. Uso de la orden `write`.

1. Cree nuevamente una variable `f` que apunte al objeto fichero en modo `'w'` con el nombre `texto.dat`

2. A continuación escriba:

```
>>> f.write('Ya es hora')
>>> f.write('de cerrar el archivo')
```

3. El cierre del fichero le dice al sistema que hemos terminado de escribir y lo deja listo para leer. Para cerrar el fichero escriba:

```
>>> f.close()
```

Ejemplo 5.4.3. Lectura del archivo creado en el ejemplo anterior.

1. Ya se puede abrir el archivo de nuevo para lectura y poner su contenido en una cadena. Esta vez el argumento de modo es 'r' para lectura. Escriba:

```
>>> f = open('texto.dat', 'r')
```

2. Escriba:

```
>>> lectura = f.read()
>>> print(lectura)
```

la respuesta será:

```
Ya es horade cerrar el archivo
```

No hay un espacio entre 'hora' y 'de' porque no se escribió un espacio entre las cadenas.

3. La función `read` también puede aceptar un argumento que le indica cuántos caracteres leer, para ello escriba:

```
>>> f = open('texto.dat', 'r')
>>> print(f.read(7))
```

la respuesta será:

```
Ya es h
```

4. Si no quedan suficientes caracteres en el archivo, `read` devuelve los que hubiera. Escriba:

```
>>> print(f.read(1000006))
```

la respuesta será:

```
orade cerrar el archivo
```

5. Cuando se llega al final del archivo, `read` devuelve una cadena vacía. Escriba:

```
>>> print(f.read())
```

Ejemplo 5.4.4. Crear un archivo de texto con tres líneas de texto separadas por saltos de línea.

1. Escriba:

```
>>> f = open('lineas.dat', 'w')
>>> f.write('línea uno\nlínea dos\nlínea tres\n')
>>> f.close()
```

2. El método `readline` lee todos los caracteres hasta e inclusive el siguiente salto de línea. Escriba:

```
>>> f = open('lineas.dat', 'r')
>>> print(f.readline())
```

la respuesta será:

```
línea uno
```

3. `readlines` devuelve todas las líneas que queden como una lista de cadenas:

```
>>> f.readlines()
['línea dos\n', 'línea tres']
```

4. Al final del archivo, `readline` devuelve una cadena vacía y `readlines` devuelve una lista vacía. Para comprobarlo escriba:

```
>>> print(f.readline())
>>> print(f.readlines())
```

El argumento de `write` debe ser una cadena, por lo que para poner otros valores en un archivo, hay que convertirlos antes en cadenas. La forma más fácil de hacerlo es con la función `str`. Una alternativa es usar el operador de formato `%` visto anteriormente.

Ejemplo 5.4.5. *Uso de la función `str`.*

1. Cree una variable `f` que apunte a un objeto fichero que se llame `num.dat` que esté preparado para escribir:

```
>>> f=open('num.dat', 'w')
```

2. Escriba en él el valor de la variable `x` de la forma:

```
>>> x = 52
>>> f.write(str(x))
```

3. Cierre el fichero y ejecute las órdenes correspondientes para su lectura tal y como mostramos anteriormente. La respuesta observada debe ser:

```
'52'
```

Cuando se crea un archivo nuevo abriéndolo y escribiendo, el nuevo archivo va al directorio en uso (aquél en el que estuviese al ejecutar el programa). Del mismo modo, cuando abre un archivo para leerlo, Python lo busca en el directorio en uso. Si quiere abrir un archivo de cualquier otro sitio, tiene que especificar la ruta del archivo, que es el nombre del directorio (o carpeta) donde se encuentra éste.

Ejemplo 5.4.6. *Para ilustrar el uso de directorios se propone abrir un archivo llamado `words` que está en un directorio llamado `dict`, que está en `share`, que está en `usr`, que está en el directorio raíz del sistema, llamado `/`. Para ello hay que escribir:*

```
>>> f = open('/usr/share/dict/words', 'r')
>>> print(f.readline())
```


Ejercicio 5.4.1. Cree un programa que genere un fichero de datos llamado `temperaturasC.dat` donde se guarden las temperaturas Celsius desde $-20\text{ }^{\circ}\text{C}$ a $40\text{ }^{\circ}\text{C}$ con un salto de $5\text{ }^{\circ}\text{C}$. Guarde el programa en un archivo con el nombre de `generaTempC.py`. Ejecute el programa y compruebe que se genera el fichero `temperaturasC.dat` de forma correcta.

Ejercicio 5.4.2. Cree un programa que lea el archivo de datos `temperaturasC.dat` y que convierta los grados Celsius en grados Fahrenheit y genere un archivo de resultados llamado `TemperaturasFC.dat` donde se guarde la tabla de temperaturas con el mismo formato que en el programa `TemperaturasFCv2.py`. Guarde este programa con el nombre de `generaTempFC.py`. Ejecute el programa y compruebe que funciona correctamente.